

Java Anonymous Classes

Concise Code & On-the-Fly Logic

codehive

codehive

What are Anonymous Classes?

An **Anonymous Class** is a nested class without a name. In Java, it allows you to declare and instantiate a class at the same time.

They are used when you need to override the behavior of a class or implement an interface for a **one-time use**, without having to create a formal .java file or a named subclass.

Key Concept: Even though they don't have a name, the Java compiler creates a .class file for them (usually named OuterClass\$1.class).

1. Extending a Class

You can create an anonymous class that extends an existing class to modify its behavior instantly.

```
// Normal Parent Class
class Animal {
    public void makeSound() {
        System.out.println("Animal sound");
    }
}

public class Main {
    public static void main(String[] args) {
        // Anonymous class defining a specific sound
        Animal dog = new Animal() {
            @Override
            public void makeSound() {
                System.out.println("Woof woof");
            }
        }; // Semicolon is mandatory!

        dog.makeSound();
    }
}
```

Output: `Woof woof`

2. Implementing an Interface

Usually, to use an interface, you must write a class that implements it. Anonymous classes allow you to implement the interface directly in the instantiation line.

```
interface Greeting {
    void sayHello();
}

public class Main {
    public static void main(String[] args) {
        // Instant implementation of the interface
        Greeting welcome = new Greeting() {
            public void sayHello() {
                System.out.println("Hello, World!");
            }
        };

        welcome.sayHello();
    }
}
```

Note: This is a very common pattern in Android development and GUI programming (like adding Button listeners).

Important Rules

Anonymous classes are powerful but come with specific constraints:

No Constructors: An anonymous class cannot have a constructor because it has no name.

Scope: It can access members of its enclosing class and local variables (if they are final or effectively final).

One Base: It can either extend ONE class OR implement ONE interface—never both at the same time.

Closing Syntax: Since anonymous class expression is part of an assignment, it must always end with a **semicolon (;)** after the closing brace.

When to use Anonymous Classes?

You should consider using an anonymous class when:

- You only need a local instance with a few method overrides.
- Creating a full separate class would be over-engineering.
- You want to pass a small piece of behavior as an argument to a method.

Mastering Java with

codehive

Keep Coding. Keep Growing.